

Bilkul. Main MediVault ko ab **simple medical-record app** nahi, balki **patient-controlled healthcare data platform** के रूप में define karunga. Neeche ek improved, complete PRD hai jo development team, college project documentation, startup pitch aur AI coding agent—sab ke liye base document ban sakta hai.

## MediVault — Product Requirements Document (PRD)

**Version:** 2.0

**Status:** Product Blueprint / Development Ready

**Product:** MediVault

**Category:** Digital Healthcare / Health Data Management

**Primary Market:** India

**Core Principle:** Your Health. Your Records. Your Control.

---

### 1. Executive Summary

**MediVault** is a secure, privacy-first digital healthcare platform that allows patients to maintain, organize, understand, and selectively share their medical information.

Instead of medical information being scattered across:

- hospitals
- clinics
- laboratories
- email
- WhatsApp
- physical documents
- different healthcare apps

MediVault provides a centralized **personal medical vault**.

The most important difference is that MediVault is designed around **patient-controlled access**.

A doctor or hospital does not automatically receive access to a patient's complete medical history.

Instead:

**Request → Purpose → Permission → Scope → Duration → Access → Audit**

Patients can:

- store medical reports
- maintain medical history
- manage prescriptions
- search doctors
- find hospitals

- book appointments
  - communicate with doctors
  - approve/reject access requests
  - grant temporary access
  - revoke access
  - monitor who accessed their information
  - create an emergency medical card
- 

## **2. Vision**

### **Vision Statement**

**Build a trusted digital healthcare ecosystem where individuals remain in control of their health information.**

MediVault aims to make healthcare information:

**Secure + Organized + Accessible + Portable + Patient-Controlled**

---

## **3. Mission**

MediVault's mission is to reduce fragmentation in healthcare information by giving patients a secure personal health-data vault and a transparent consent system.

---

## **4. Core Product Philosophy**

MediVault follows five principles:

### **1. Patient Ownership**

The patient is the primary controller of their personal medical information.

### **2. Explicit Consent**

Medical information should only be shared according to valid authorization.

### **3. Least Privilege**

A healthcare professional should receive only the information necessary for the authorized purpose.

### **4. Time-Bound Access**

Access should expire automatically when its approved duration ends.

### **5. Transparency**

Patients should be able to see who requested, viewed, or accessed their information.

---

## 5. Problem Statement

Healthcare information is fragmented.

A patient may have:

Hospital A



Blood Reports

Clinic B



Prescriptions

Diagnostic Lab



MRI/CT Reports

Personal Phone



Photos/PDFs

Physical Files



Old Medical Records

This creates problems:

- lost reports
- duplicate tests
- difficult record sharing
- poor medical history continuity
- lack of patient visibility
- uncontrolled document sharing
- emergency information gaps

MediVault attempts to solve this through a unified digital health vault.

---

## **6. Target Users**

### **Primary Users**

#### **Patient**

The central user.

Needs:

- secure record storage
- medical history
- doctor discovery
- appointments
- controlled sharing
- emergency information

#### **Doctor**

Needs:

- patient-authorized medical information
- appointment management
- prescriptions
- consultation notes
- communication

#### **Hospital**

Needs:

- doctors
- departments
- appointments
- authorized patient records
- organization management

#### **Admin**

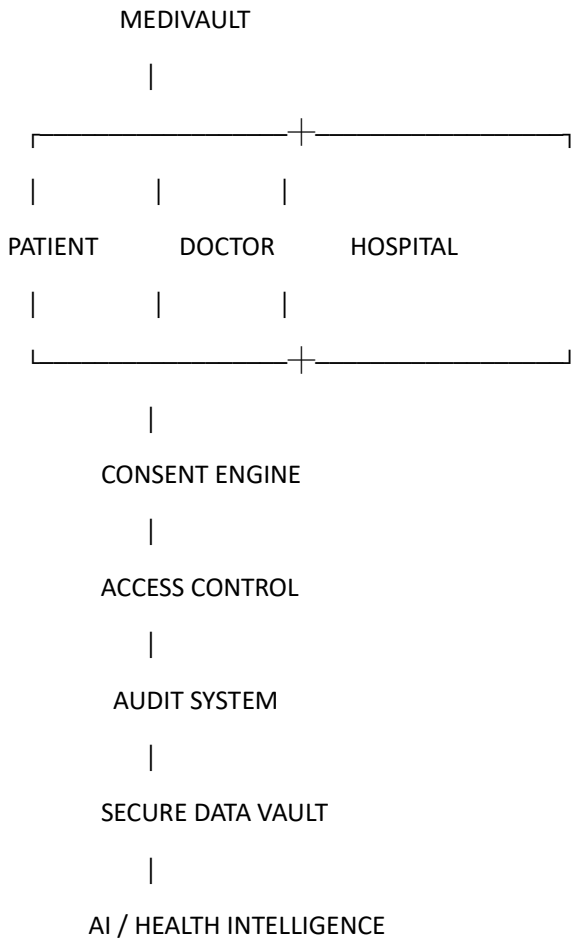
Needs:

- platform management
- user verification
- security monitoring

- organization management
- system analytics

Admin must **not** automatically access private patient medical records.

7. Product Ecosystem



8. User Roles & Permissions

| Feature                | Patient | Doctor     | Hospital   | Admin   |
|------------------------|---------|------------|------------|---------|
| Manage own profile     | ✓       | ✓          | ✓          | Limited |
| Upload own reports     | ✓       | Authorized | Authorized | ✗       |
| View own records       | ✓       | Authorized | Authorized | ✗       |
| Request patient access | —       | ✓          | ✓          | ✗       |

| Feature             | Patient | Doctor       | Hospital              | Admin             |
|---------------------|---------|--------------|-----------------------|-------------------|
| Approve access      | ✓       | ✗            | ✗                     | ✗                 |
| Revoke access       | ✓       | ✗            | ✗                     | ✗                 |
| Create prescription | ✗       | ✓            | Authorized            | ✗                 |
| Manage appointments | ✓       | ✓            | ✓                     | Limited           |
| View audit history  | Own     | Own activity | Organization activity | Security metadata |
| Manage users        | ✗       | ✗            | Limited               | ✓                 |

---

## 9. Identity Architecture

MediVault must never use Aadhaar as the application's primary identifier.

Architecture:

Identity Verification



Verification Result



Internal MediVault Identity



Patient ID



Application Database

Example:

MV-PAT-8F29K1

The MongoDB internal identifier should remain separate from the user-facing Patient ID.

---

## 10. Aadhaar Verification — Demo Mode

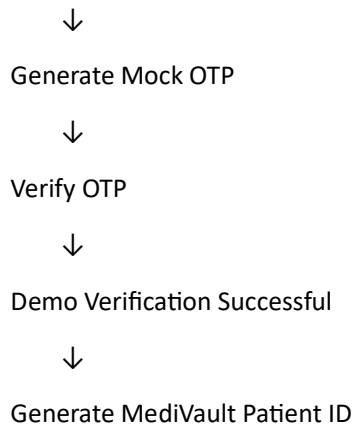
The prototype must **not implement fake or unauthorized real Aadhaar authentication**.

The application should explicitly display:

### Aadhaar Verification — Demo Mode

Development flow:

Enter Demo Identity Details

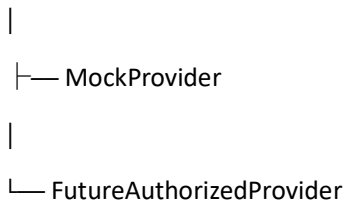


Important rules:

- Do not claim UIDAI authentication.
- Do not store OTPs permanently.
- Do not use Aadhaar number as Patient ID.
- Do not claim ABDM certification.
- Keep identity provider behind an abstraction layer.

Future architecture:

IdentityVerificationProvider



---

## 11. Patient Registration

Create a five-step registration wizard.

### Step 1 — Personal Information

- Full name
- Mobile
- Email
- Date of birth
- Gender

### Step 2 — Identity

- Demo verification

- Mock OTP
- Verification status

### **Step 3 — Account**

- Username
- Password
- Confirm password

### **Step 4 — Health & Emergency**

- Blood group
- Emergency contact
- Address
- Profile photo
- Allergies
- Optional health information

### **Step 5 — Privacy & Consent**

- Terms
  - Privacy policy
  - Data processing consent
  - Consent version
  - Timestamp
- 

## **12. Authentication**

Support:

- Patient ID login
- Username login
- Email login
- Password
- Forgot password
- Reset password
- Logout
- Logout from all devices
- Session management

- Optional MFA

Architecture:

Access Token

+

Refresh Token

+

Session Management

Use:

- Argon2id
  - HTTP-only cookies where appropriate
  - refresh-token rotation
  - token revocation
- 

### 13. Patient Dashboard

Dashboard should provide a quick health overview.

#### Main sections

Welcome back

Health Summary

Upcoming Appointment

Active Medications

Recent Reports

Pending Access Requests

Active Consents

Security Status

Emergency Card

Medical Timeline

---

### 14. Medical Profile

Sections:

**Personal**

- Name
- DOB
- Gender
- Contact
- Address

**Medical**

- Blood group
- Allergies
- Conditions
- Medications
- Surgeries
- Family history
- Vaccinations

**Emergency**

- Emergency contact
- Relationship
- Phone

Sensitive updates should generate audit events.

---

**15. Medical Vault**

The vault is MediVault's central storage system.

Users can organize:

Medical Reports

Prescriptions

Lab Results

Imaging

Discharge Summaries

Vaccination Records

Medical Certificates

Consultation Documents

Other

Every document contains metadata.

Example:

Document ID

Patient ID

Category

Title

Document Date

Uploaded Date

Doctor

Hospital

Description

Storage Key

File Type

File Size

Checksum

---

## **16. Secure File Storage**

Medical files must be private.

Architecture:

Frontend



Backend



Authorization Check



Consent Check



Private Storage



Short-Lived Access

Development:

**Private local storage**

Production:

**Private S3-compatible object storage**

Never expose medical documents using permanent public URLs.

---

## 17. Medical Timeline

Convert records into a chronological timeline.

Example:

2026

January 12

 Doctor Consultation

January 15

 Blood Test

January 18

 Prescription

February 02

 Hospitalization

February 15

 MRI Report

Users can filter by:

- date
  - doctor
  - hospital
  - record type
-

## 18. Doctor Discovery

Patients can search doctors by:

- specialty
- medical problem
- location
- availability
- consultation type

Doctor profile:

- Name
  - Specialty
  - Qualifications
  - Experience
  - Hospital
  - Languages
  - Location
  - Consultation mode
  - Availability
  - Verification status
- 

## 19. Hospital Discovery

Search nearby hospitals based on:

- location
- department
- specialty
- emergency availability
- distance

Maps must use an abstraction:

MapServiceProvider

Development:

MockMapServiceProvider

Production:

Google Maps / equivalent

---

## **20. Appointment Management**

Patients can:

- book
- cancel
- reschedule
- view upcoming appointments
- view appointment history

Appointment states:

REQUESTED

CONFIRMED

RESCHEDULED

COMPLETED

CANCELLED

NO\_SHOW

Double booking must be prevented at backend/database level.

---

## **21. Doctor Dashboard**

Doctor dashboard:

Today's Appointments

Pending Access Requests

Authorized Patients

Recent Consultations

Prescriptions

Messages

Doctor can:

- manage availability
- view appointments
- request records
- view approved records

- create prescriptions
- write consultation notes
- message patients

---

## 22. Patient-Controlled Consent System

This is the **core MediVault feature**.

### Access workflow

Doctor



Requests Access



Selects Purpose



Selects Required Records



Selects Duration



Patient Notification



Patient Reviews



Approve / Reject



Consent Created



Limited Access



Audit



Automatic Expiry

---

### 23. Access Request

Example:

Dr. Rahul Sharma

Cardiologist

Purpose:

Cardiology Consultation

Requested:

✓ Blood Reports

✓ ECG

✓ Prescriptions

Duration:

7 Days

[Approve]

[Reject]

---

### 24. Consent Scope

Available scopes:

PROFILE

MEDICAL\_HISTORY

LAB\_REPORTS

IMAGING

PRESCRIPTIONS

MEDICATIONS

ALLERGIES

VACCINATIONS

EMERGENCY\_INFORMATION

A doctor should never automatically receive all scopes.

---

## **25. Temporary Access**

Patient can grant:

Doctor

Purpose

Records

Duration

Example:

Dr. Sharma can access ECG and blood reports for 72 hours for cardiology consultation.

Backend must enforce expiry.

---

## **26. Access Revocation**

Patient can revoke active access immediately.

Active Consent

↓

Revoke

↓

Consent = REVOKED

↓

Future Requests = DENIED

Frontend hiding is not sufficient.

Backend authorization must reject future requests.

---

## **27. "Who Has Access?" Dashboard**

Create a dedicated screen:

### **My Data Access**

Dr. Rahul Sharma

Cardiology

✓ ECG

✓ Blood Reports

Expires:

21 September 2026

[Revoke Access]

This should be one of MediVault's signature screens.

---

## **28. Emergency Medical Card**

Create a digital emergency card.

Possible information:

Name

Blood Group

Allergies

Current Medications

Emergency Contact

Generate a QR code that points to a secure MediVault endpoint.

Do **not** embed private medical records directly in the QR code.

QR flow:

QR



Secure MediVault endpoint



Emergency verification



Limited information

---

## **29. Emergency / Break-Glass Access**

For genuine emergency scenarios:

Healthcare Provider



Emergency Access Request



Emergency Reason



Limited Scope



Short Expiration



Access



Audit



Patient Notification

Every emergency access must be highly visible in the audit system.

---

### **30. Prescriptions**

Doctors can create:

- diagnosis/context
- medicine
- dosage
- frequency
- duration
- instructions
- notes

Patients can view their prescriptions.

---

### **31. Messaging**

Provide secure patient-doctor communication.

Features:

- conversations
- unread messages

- timestamps
- appointment context
- controlled attachments

Messaging should follow authorization rules.

---

### **32. Notifications**

Notification types:

Access Request

Access Approved

Access Rejected

Access Expiring

Access Expired

Appointment Confirmed

Appointment Reminder

Prescription Created

Report Uploaded

Security Alert

---

### **33. Security Center**

Create:

#### **Account Security**

Display:

Password

MFA

Active Sessions

Recent Logins

Active Consents

Connected Providers

Security Alerts

Actions:

- Change password

- Enable MFA
  - Logout all devices
  - Revoke sessions
  - Review activity
- 

### **34. Audit System**

Every sensitive event must be logged.

Examples:

Patient uploaded report

Doctor requested access

Patient approved access

Doctor viewed report

Patient revoked access

Doctor created prescription

Patient downloaded report

Login detected

Password changed

Emergency access used

Audit fields:

actorId

actorRole

patientId

action

resourceType

resourceId

purpose

timestamp

requestId

result

---

### **35. AI / ML Layer**

AI should be a **supporting intelligence layer**, not an AI doctor.

### **AI Medical Record Assistant**

Capabilities:

#### **Document Summarization**

Upload:

Blood\_Report.pdf

AI generates a simple explanation.

#### **Natural Language Search**

Patient asks:

"Show me my blood reports from 2025."

System finds relevant records.

#### **Medical Timeline Intelligence**

AI can summarize:

"You have 4 documented consultations and 3 lab reports in the selected period."

### **RAG**

Future architecture:

Medical Documents



OCR / Parser



Chunking



Embeddings



Vector Store



Patient-specific RAG



AI Assistant

The AI must respect authorization boundaries.

---

### **36. AI Safety**

The system must not claim to diagnose patients.

Avoid:

"You have diabetes."

Prefer:

"Your uploaded report contains a glucose value of X. Please discuss the result with a qualified healthcare professional."

Clearly label AI-generated content.

---

### **37. Health Insights**

Provide non-diagnostic insights:

Reports uploaded this year

Medication count

Appointment frequency

Recorded health trends

Upcoming follow-up

Missing profile information

Do not present statistical trends as medical diagnoses.

---

### **38. Admin Dashboard**

Admin modules:

Overview

Users

Doctors

Hospitals

Organizations

Verification

Security Events

System Audit

Reports

Platform Settings

Admin should not have unrestricted access to patient medical data.

---

### **39. Organization Model**

Hospital architecture should eventually support:

Organization



Hospital



Department



Doctor



Patient Relationship

This allows MediVault to scale beyond individual doctors.

---

### **40. Database Architecture**

Core MongoDB collections:

users

patientProfiles

doctorProfiles

hospitals

organizations

medicalRecords

medicalDocuments

prescriptions

appointments

accessRequests

consents

notifications

conversations

messages

sessions

auditLogs

verificationSessions

emergencyAccess

Important indexes should be created for:

- patientId
- email
- mobile
- specialty
- location
- appointment date
- consent status
- consent expiry

---

#### **41. API Architecture**

Use REST APIs.

Major groups:

/auth

/patients

/doctors

/hospitals

/appointments

/medical-records

/documents

/prescriptions

/access-requests

/consents

/messages

/notifications

/security

/admin

All sensitive endpoints require authentication and authorization.

---

## 42. Central Authorization Engine

Create:

MedicalAccessService

Conceptually:

Request



Authentication



RBAC



Resource Ownership



Consent



Scope



Purpose



Expiry



Emergency Rule



ALLOW / DENY



Audit

This should be one of the most important backend services.

---

## 43. Security Architecture

Use:

- Argon2id
- JWT
- Refresh token rotation
- HTTP-only cookies
- Helmet
- CORS
- Rate limiting
- Zod validation
- request IDs
- secure headers
- private file storage
- short-lived document access
- audit logs
- environment secrets
- secure error handling

Never expose:

- passwords
- OTPs
- refresh tokens
- API secrets
- unnecessary medical information

---

#### **44. Privacy Architecture**

Separate:

Identity Data

Account Data

Medical Data

Consent Data

Audit Data

Follow data minimization.

Avoid unnecessary sensitive data in:

- URLs
  - logs
  - analytics
  - error messages
  - frontend state
- 

#### **45. India Compliance Direction**

MediVault should be designed with India's healthcare and privacy ecosystem in mind.

Potential future integrations should consider:

- India's applicable privacy/data-protection requirements
- healthcare-data handling requirements
- authorized identity-verification mechanisms
- ABDM ecosystem
- applicable consent and interoperability standards

The prototype should explicitly state:

**MediVault is a prototype and is not ABDM-certified.**

Compliance and certification should be treated as a separate production workstream involving appropriate legal, security, and healthcare-domain review.

---

#### **46. Provider Abstraction Architecture**

External services should not be tightly coupled to the application.

Create:

IdentityVerificationProvider

OtpProvider

EmailProvider

SmsProvider

MapsProvider

MedicalFileStorage

HealthcareInteroperabilityProvider

Development implementations:

MockIdentityProvider

MockOtpProvider

MockEmailProvider

MockSmsProvider

MockMapsProvider

LocalPrivateStorage

MockHealthcareProvider

Production implementations can later be added without rewriting the core business logic.

---

## **47. Frontend Design System**

Visual identity:

**Brand**

**MediVault**

**Suggested tagline**

**Your Health. Your Records. Your Control.**

**Design direction**

- professional
- calm
- trustworthy
- modern
- minimal
- accessible

Avoid making it look like a generic hospital website.

---

## **48. Main Navigation**

Patient:

Dashboard

Medical Profile

Medical History

Medical Vault

Prescriptions

Medications

Doctors  
Hospitals  
Appointments  
Access & Consent  
Messages  
Emergency Card  
Security  
Settings

Doctor:

Dashboard  
Appointments  
Patients  
Access Requests  
Authorized Records  
Prescriptions

Messages

Profile

Security

Hospital:

Dashboard

Doctors

Departments

Appointments

Patients

Access

Reports

Organization

Settings

Admin:

Dashboard

Users

Doctors

Hospitals

Organizations

Verification

Security

Audit

Settings

---

#### **49. UX Requirements**

Every async action needs:

Loading

Success

Error

Empty

Retry

Every destructive action needs confirmation.

Example:

Are you sure you want to revoke this doctor's access?

---

#### **50. Responsive Design**

MediVault must work on:

- mobile
- tablet
- laptop
- desktop

No:

- overlapping cards
- clipped text
- broken tables
- horizontal overflow
- inaccessible buttons

- broken charts
  - crowded navigation
- 

## **51. Accessibility**

Implement:

- semantic HTML
  - keyboard navigation
  - visible focus states
  - accessible labels
  - adequate contrast
  - screen-reader support
  - proper form errors
  - accessible dialogs
- 

## **52. Non-Functional Requirements**

### **Performance**

Target:

- fast initial load
- lazy-loaded pages
- optimized images
- paginated medical records
- efficient API queries

### **Reliability**

- graceful error handling
- retries where appropriate
- database indexes
- transaction/consistency handling where needed

### **Security**

Security takes priority over convenience.

### **Scalability**

Architecture should allow:

1,000 users

→ 10,000

→ 100,000

→ 1M+

without requiring a complete rewrite.

---

## **53. Testing Strategy**

### **Unit Testing**

Test:

- authorization
- consent
- validation
- token handling
- business logic

### **Integration Testing**

Test:

- registration
- login
- medical upload
- appointment booking
- access requests
- consent
- prescriptions

### **Security Testing**

Critical scenarios:

Doctor → Patient B's record

→ DENIED

Expired Consent

→ DENIED

Revoked Consent

→ DENIED

Admin → Patient medical record

→ DENIED

Patient A → Patient B's report

→ DENIED

---

## 54. MVP Definition

The first production-style MVP should contain:

### Patient

- Registration
- Demo verification
- Login
- Dashboard
- Medical profile
- Medical vault
- Medical history
- Timeline
- Doctor search
- Hospital search
- Appointment booking
- Consent
- Temporary access
- Revocation
- Emergency card
- Security center

### Doctor

- Registration
- Profile

- Availability
- Appointments
- Access requests
- Authorized records
- Prescriptions
- Messaging

#### **Platform**

- RBAC
  - Consent engine
  - Audit system
  - Private file storage
  - Notifications
  - Security controls
- 

#### **55. V2 Features**

After MVP:

AI Document Summarization

OCR

Medical RAG

Natural Language Search

Health Insights

Advanced Emergency Access

Hospital Organization Management

Advanced Analytics

---

#### **56. V3 Features**

Potential long-term ecosystem:

ABDM integration

Healthcare interoperability

Authorized identity providers

Lab integrations

Hospital integrations

Wearable integrations

Telemedicine

Insurance integrations

Pharmacy integrations

These should come only after the core consent and security architecture is mature.

---

## **57. Features Explicitly Out of Scope for MVP**

Do not initially build:

- online pharmacy
  - insurance marketplace
  - blockchain
  - AI diagnosis
  - real Aadhaar authentication
  - unrestricted admin medical access
  - IoT medical hardware
  - complex payment system
  - full telemedicine ecosystem
- 

## **58. Key Success Metrics**

### **Product Metrics**

- registered patients
- verified doctors
- uploaded medical documents
- appointments booked
- access requests
- consent approval rate
- consent revocation rate
- active users

### **Security Metrics**

- unauthorized access attempts blocked

- expired access correctly denied
- revoked access correctly denied
- audit events generated
- suspicious sessions detected

#### UX Metrics

- registration completion
- appointment completion
- report upload success
- consent decision completion

---

### 59. Core User Journey

Patient



Register



Demo Identity Verification



MediVault Patient ID



Complete Profile



Upload Medical Records



Search Doctor



Book Appointment



Doctor Requests Records



Patient Reviews Request



Patient Approves Specific Records



Doctor Gets Limited Access



Doctor Consults Patient



Prescription Created



Patient Receives Prescription



Consent Expires



Access Automatically Revoked



Patient Reviews Audit History

---

## 60. MediVault's Strongest Differentiators

If you're presenting this project in a college viva, hackathon, interview, or startup pitch, focus on these:

### 1. Patient-Controlled Medical Data

The patient decides who can access what.

### 2. Temporary Consent

Access automatically expires.

### 3. Instant Revocation

Patients can revoke access.

### 4. Complete Auditability

Patients can see access activity.

### 5. Emergency Medical Card

Critical information can be securely accessed during emergencies.

### 6. AI Medical Record Assistant

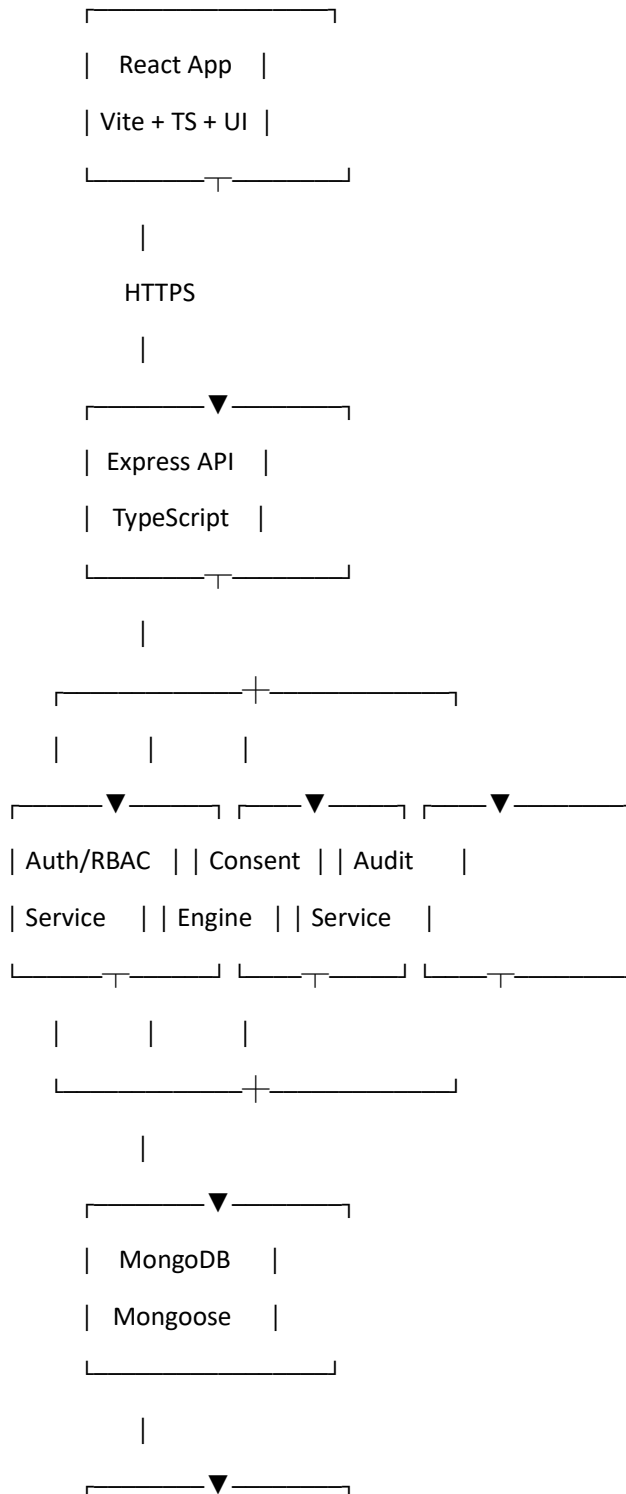
AI helps patients understand and search their records without positioning itself as a doctor.

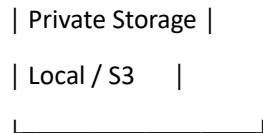
## 🌱 7. Future Interoperability

Architecture is prepared for authorized healthcare ecosystem integrations.

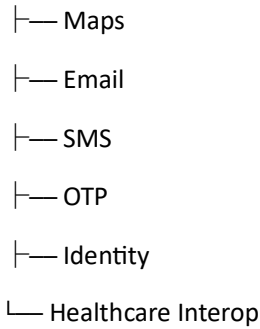
---

### 61. Recommended Architecture





#### External Provider Abstractions



## 62. Final Product Positioning

### One-line pitch

**MediVault is a patient-controlled digital healthcare vault that securely organizes medical records and lets patients decide who can access specific health information, for what purpose, and for how long.**

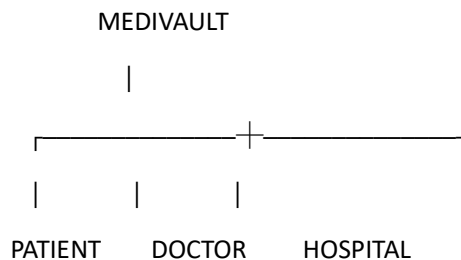
### 30-second pitch

**Today, our medical information is scattered across hospitals, laboratories, clinics, phones, and physical files. MediVault brings these records into one secure digital vault while keeping the patient in control. Doctors and hospitals can request specific information for a specific purpose and duration, but access is granted only with appropriate authorization and is fully auditable. Patients can revoke access, manage their medical history, book appointments, receive prescriptions, and create an emergency medical card.**

### My recommendation for your actual project

**Don't build every feature in this PRD simultaneously.**

Build MediVault around this **core 6-module architecture first**:





 CONSENT ENGINE



 MEDICAL VAULT



 AUDIT SYSTEM



 AI LAYER

**The Consent Engine + Secure Medical Vault + Audit System should be your technical centerpiece.** AI should come after these three are working properly. That combination gives MediVault a much stronger identity than simply building another doctor-booking or medical-report-storage website.